

PCB-Inspector

PCB 동박 결함 자동 검사 — 개발 백서

기획 · 설계 · 구현 · 배포 · 검증(mAP 재현) · 출시

이재환 · 2026. 06. 12 · v1.0

목차

목차.....	2
1. 프로젝트 개요.....	3
2. 기획 (Planning).....	4
2.1 문제 정의.....	4
2.2 요구사항.....	4
2.3 데이터셋 — DsPCBSD+	4
3. 설계 (Design).....	5
3.1 검사 파이프라인	5
3.2 이중 임계값 판정 (핵심 설계).....	5
3.3 모델·추론 설계.....	6
4. 구현 (Implementation)	7
4.1 추론 엔진 (inference.py)	7
4.2 데스크톱 GUI (main.py · PySide6)	7
4.3 모델 변환 (export_onnx.py)	7
5. 배포 (Deployment).....	8
5.1 실행.....	8
5.2 패키징	8
5.3 크로스플랫폼	8
6. 검증 (Validation) — mAP 재현.....	9
6.1 종합 지표.....	9
6.2 클래스별 mAP@50.....	9
6.3 혼동 행렬 (Confusion Matrix).....	10
6.4 실행 검증 — 실제 검출	10
7. 출시 · 한계 · 다음.....	12
7.1 현재 상태	12
7.2 한계와 보완	12
8. 회고 및 교훈.....	12

1. 프로젝트 개요

한 줄 요약: PCB 사진을 열면 9종 미세 결함을 자동 검출하고, confidence 이중 임계값으로 자동 불량 / 재검 필요 / 통과를 판정하는 양산 AOI(자동광학검사) 보조 데스크톱 앱. 공개 데이터셋으로 직접 파인튜닝했고, 검증셋 2,051장으로 mAP@50 0.843을 재현했다.

양산 라인의 PCB 표면 검사는 숙련 검사원이 9종의 미세 결함을 육안으로 판정한다. 피로 누적과 결함 형태의 미묘함 때문에 오판·누락이 발생한다. PCB-Inspector는 이 판정을 보조하는 도구로, 단순히 '검출'에 그치지 않고 검사원의 책임을 분리하는 판정 설계(이중 임계값)에 초점을 둔다.

항목	내용
한 줄 정의	PCB 사진 → 9종 결함 자동 검출 + 자동불량/재검/통과 판정
모델	YOLO26s 파인튜닝 (Ultralytics) · 9.47M 파라미터 · 20.5 GFLOPs
데이터	DsPCBSD+ — 10,259장 / 결함 20,276개 / 9 클래스 (Roboflow-Figshare 공개)
추론	ONNX Runtime (CPU) — end2end 출력 (1,300,6) · 자체 NMS
성능	mAP@50 0.843 (검증셋 2,051장 재현) · CPU 약 140ms/장
배포	PySide6 데스크톱 앱 · Windows .exe / macOS .app 빌드

역량 구분(정직 표기): 사전학습 모델 활용이 아니라 도메인 데이터(DsPCBSD+)로 직접 파인튜닝한 카드다. 가중치(best.pt)·변환본(best.onnx)을 포함하고, 본 문서의 모든 수치는 검증셋으로 재현 가능하다.

2. 기획 (Planning)

2.1 문제 정의

- 양산 PCB 검사원이 9 종 미세 결함(단락·스퍼·잔여구리·단선·마우스바이트·홀 브레이크아웃·스크래치·도체이물·기판이물)을 육안 판정 → 피로·오판.
- 전수 자동 판정만 하면 오탐(false positive)으로 양품을 버리는 비용이 크다 → '확실한 불량만 자동, 애매한 건 사람'이라는 판정 설계가 필요.

2.2 요구사항

요구	내용
검출 정확도	9종 결함, 실용 가능한 mAP (목표 0.8+)
판정 설계	자동 불량은 고신뢰(precision 우선), 애매한 건 검사원 재검
배포 용이성	GPU 없는 검사 PC에서 동작, 파이썬 설치 없이 실행 가능
검사원 UX	폴더 일괄 검사 · 확대 확인 · 임계값 실시간 조정 · CSV 내보내기

2.3 데이터셋 — DsPCBSD+

- Guangzhou FastPrint 실제 PCB 생산 라인 이미지 · Nature Scientific Data(2024) 게재 · YOLO/COCO 포맷.
- 9 클래스를 원인·위치·형태로 분류: 동박 잔류 / 동박 결핍 / 도체 스크래치 / 이물 4 대 범주 → 9 세부 클래스.

3. 설계 (Design)

3.1 검사 파이프라인

PCB 이미지 1장이 판정에 이르는 경로를 단일 파이프라인으로 설계했다. 무거운 전처리 없이, letterbox로 비율을 유지한 채 640으로 맞추고 ONNX 추론 → 좌표 복원 → 판정으로 이어진다.

검사 파이프라인 — PCB 사진 1장이 판정까지 가는 길

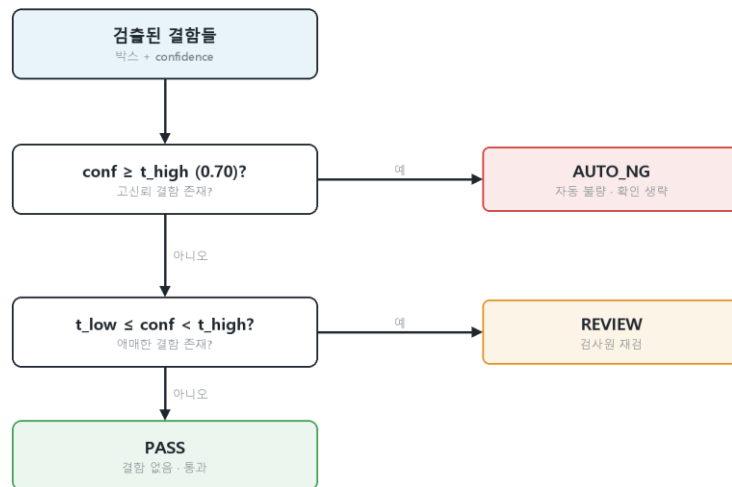


[그림 1] 검사 파이프라인 — 5단계

3.2 이중 임계값 판정 (핵심 설계)

단일 임계값은 '불량/양품'만 나눈다. PCB-Inspector는 두 임계값(t_{low} - t_{high})으로 세 구간을 만들어, 고신뢰 결함만 자동 불량으로 처리하고 애매한 구간은 검사원에게 넘긴다. 자동화의 책임 범위를 명확히 한 설계다.

이중 임계값 판정 — 자동 불량은 고신뢰만, 애매하면 사람에게



[그림 2] 이중 임계값 판정 로직

구간	판정 / 의미
$conf \geq t_{high}$ (기본 0.70)	AUTO_NG — 자동 불량, 사람 확인 생략 (precision 우선 구간)
$t_{low} \leq conf < t_{high}$	REVIEW — 검사원이 확인
$conf < t_{low}$ (기본 0.25)	보고 안 함 — 노이즈 억제

3.3 모델·추론 설계

- YOLO26s 채택 — n/s/m 중 정확도·속도 트레이드오프에서 양산 PC(CPU)에 적합한 s 선택.
- ONNX end2end export — NMS 가 그래프에 포함된 (1,300,6) 출력과 raw (1,4+nc,N) 출력을 추론 엔진이 모두 처리.
- CPU 추론 — GPU 의존 제거로 검사 PC 에 그대로 배포.

4. 구현 (Implementation)

4.1 추론 엔진 (inference.py)

- letterbox → ONNX 추론 → 출력 형태 자동 분기(end2end / raw) → 좌표 역변환 → 클램핑 → confidence 정렬.
- raw 출력 경로에는 자체 NMS(IoU 0.5) 구현 — 외부 의존 없이 박스 정리.
- verdict(dets, t_low, t_high) → AUTO_NG / REVIEW / PASS 이미지 단위 판정.

4.2 데스크톱 GUI (main.py · PySide6)

영역	기능
좌측	파일별 판정 리스트 — 색상으로 AUTO_NG/REVIEW/PASS 구분
중앙	박스 오버레이 — 휠 줌·드래그로 결함 부위 확대 확인
우측	t_low/t_high 슬라이더(실시간 반영) · 결함 테이블 · CSV 내보내기

4.3 모델 변환 (export_onnx.py)

- best.pt → best.onnx (imgsz 640, simplify) — Ultralytics export. 변환본 36MB.

5. 배포 (Deployment)

5.1 실행

- 의존성: onnxruntime · opencv-python · numpy · PySide6 (requirements.txt).
- 실행: `python app/main.py` (models/best.onnx 필요).

5.2 패키징

- Windows: `build_exe.bat` (PyInstaller) → dist/PCB-Inspector/ zip 배포 → 파이썬 없이 .exe 더블클릭.
- macOS: `build_app_mac.sh` → .app 빌드.

5.3 크로스플랫폼

`setup_mac.sh`로 환경 구성 + ONNX 변환을 한 번에 처리. ONNX(CPU) 추론이라 OS·GPU에 독립적으로 동일 동작.

6. 검증 (Validation) — mAP 재현

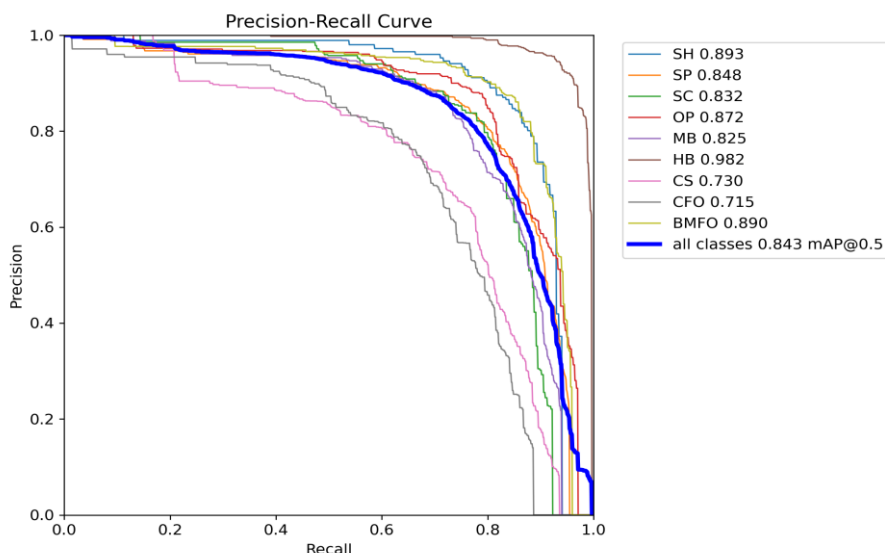
README의 mAP@50 0.843은 주장이 아니라 재현 가능한 수치다. 2026-06-12, DsPCBSD+ 검증 셋 2,051장(결함 4,092개)으로 best.pt를 직접 재검증해 동일 수치를 확인했다. 산출 차트·CSV는 docs/metrics/에 포함.

6.1 종합 지표

지표	값	비고
mAP@50	0.843	주장치 0.843과 일치
mAP@50-95	0.503	엄격 IoU 평균
Precision	0.836	
Recall	0.781	
검증 규모	2,051장 / 4,092 결함	DsPCBSD+ val

6.2 클래스별 mAP@50

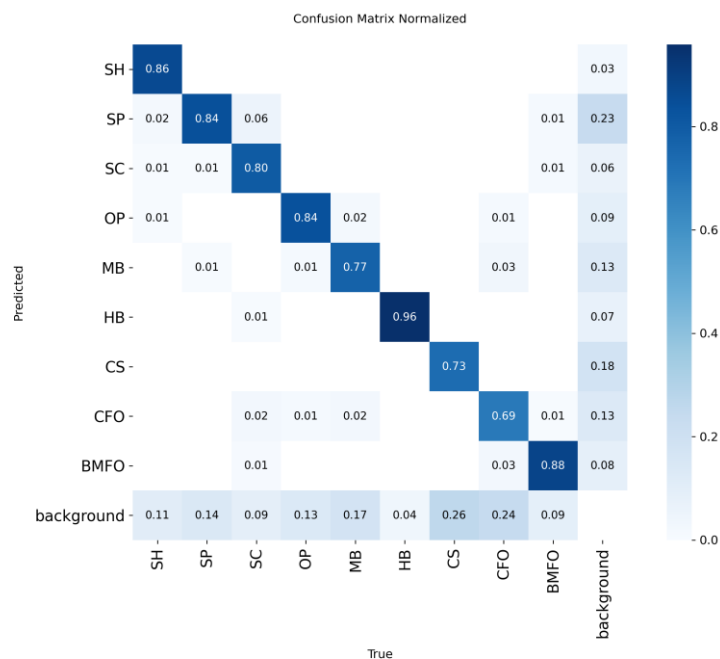
클래스	mAP@50	해석
HB 홀 브레이크 아웃	0.982	형태가 뚜렷 — 최고 성능
SH 단락	0.893	
BMFO 기판 이물	0.890	
OP 단선	0.872	
SP 스퍼	0.848	
SC 잔여구리	0.832	
MB 마우스바이트	0.825	
CS 스크래치	0.730	형태 다양 — 상대적 약점
CFO 도체 이물	0.715	배경 혼동 — 보강 대상



[그림 3] 9클래스 Precision-Recall 곡선 (all 0.843 mAP@50)

6.3 혼동 행렬 (Confusion Matrix)

대각선(정분류)이 0.69~0.96으로 강하게 형성된다. 오분류는 주로 background 혼동(CFO·CS) — 미세 이물이 배경과 헷갈리는 도메인 난이도를 반영한다.



[그림 4] 정규화 혼동 행렬

6.4 실행 검증 — 실제 검출

검증셋 12장에 ONNX 추론을 돌려 실제 검출·판정·CSV 생성을 확인했다. OP 0.91·HB 0.95·CFO 0.90 등 정확히 박싱. 12장 중 AUTO_NG 9 · REVIEW 3 · PASS 0, 평균 약 140ms/장(CPU).



[그림 5] 실검출 예 — OP 0.91·MB 0.75·BMFO 0.56 (AUTO_NG)

7. 출시 · 한계 · 다음

7.1 현재 상태

학습·변환·검증·데스크톱 배포까지 end-to-end 완성. 검증 수치가 재현되며 .exe/.app 빌드로 검사 PC에 즉시 배포 가능한 단계.

7.2 한계와 보완

영역	현재	다음
약한 클래스	CFO 0.715 · CS 0.730	해당 클래스 데이터 증강·하드샘플 마이닝
속도	CPU 약 140ms/장	INT8 양자화 · 배치 추론으로 라인 속도 대응
임계값	고정 기본값(0.25/0.70)	라인별 비용 기반 임계값 자동 보정
MES 연동	CSV 내보내기	검사 결과 DB·MES 직접 연동

8. 회고 및 교훈

- **판정 설계가 모델 정확도만큼 중요하다** — 이중 임계값으로 자동화 책임을 분리한 것이 양산 적용의 핵심. 100% 자동이 아니라 '확실한 것만 자동'이 현장에서 신뢰받는다.
- **수치는 재현 가능해야 한다** — mAP 0.843 을 주장으로 두지 않고 검증셋으로 재현해 차트까지 남겼다. 면접에서 '직접 했다'를 증명하는 가장 확실한 방법.
- **약점을 숨기지 않는다** — CFO·CS 가 약한 것은 결함 형태의 본질적 난이도. 혼동 행렬로 드러내고 보강 방향을 명시하는 것이 더 신뢰를 준다.
- **배포 형태가 가치를 완성한다** — ONNX(CPU) + .exe 빌드로 'GPU 없는 검사 PC 에 그대로'가 되어야 데모를 넘어 도구가 된다.